

PROVENANCE

AI Systems Architecture Review

Verifiable, tenant-isolated LLM inference controls for regulated AI — reproducibility receipts, and a measured cross-tenant cache-leak fix



Release/ship date · 12 September 2026 · Roshan Rana, AI Systems Architect

Executive summary

What it is

- Two working controls for a bank's shared LLM inference platform: ATTEST proves a model's output is reproducible enough for a validator (SR 11-7); BARRIER closes a routing-layer channel that leaks one tenant's prompts to another.
- Both ship against real vLLM, SGLang and llm-d — not a mock of them.

What it proves

- Batched inference at temperature 0 is not reproducible: 34 of 128 identical requests returned distinct logprob vectors. Determinism costs 18.0% of throughput isolated from caching (SGLang), 22.7% confounded with it (vLLM).
- The routing-index leak is real (AUC 1.0000, $p=9.999e-05$) and the tenant-salt fix closes it (AUC 0.5000, at chance) — same schedule, same pre-registered rule, measured in public CI on every push for £0.

Why it is enterprise-ready

- A gated lifecycle (phases 0–7), 12 ADRs and a 27-entry findings log — four of them guards that contained the defect they were written to catch, kept rather than hidden.
- 332 Python + Go tests; the statistics are implemented and unit-tested in-house so a reviewer can read the decision rule rather than trust a library call.

Ask of the audience: agree the pilot scope for the two open measurements — a client-observable timing oracle on real vLLM (FR-B-09) and a signed receipt against a live engine — and the GPU budget for the next session.

The problem and the users

The model-risk problem

SR 11-7 and its analogues assume a model's output can be reproduced. On a default vLLM deployment the honest answer is that nobody can: batched inference is not deterministic even at temperature 0, because GPU kernels pick different reduction orders depending on what else is in the batch.

The information-barrier problem

llm-d's cache-aware router makes a shared platform fast by routing matching prompt prefixes to the pod that already holds them. In a bank the prefixes are the sensitive part — by default, two tenants on one model share one routing namespace.

Actor	Needs	Frequency
Model validator / auditor	Demonstrable reproducibility for a specific output, model identity anchored outside the platform team's control	On review
Platform / SRE team	A mitigation that ships as a plugin, not a fork, and a diff that states exactly what changed	Deploy / operate
Compliance / InfoSec	Evidence the routing layer cannot leak across an information barrier, and an honest account of what was and was not measured	Ongoing
Tenants (Research vs M&A)	The same shared platform, and a guarantee neither can observe what the other is asking	Every request

Solution at a glance



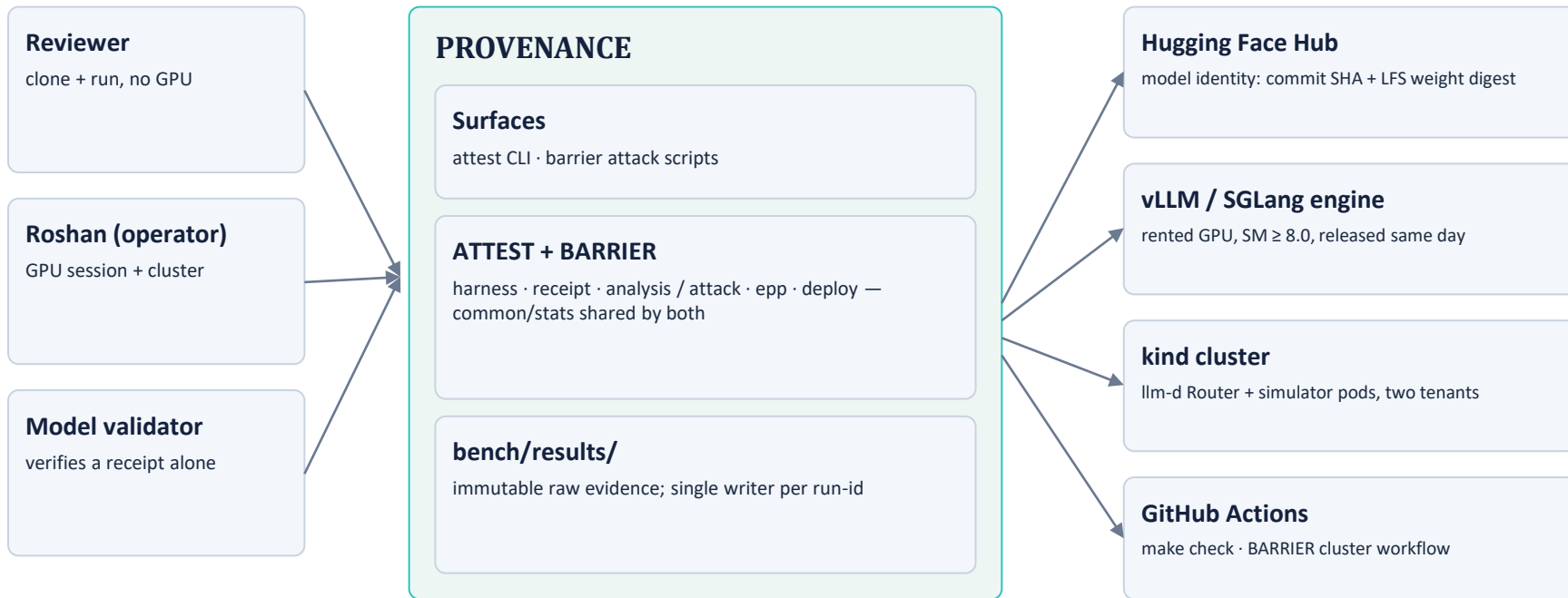
Surfaces

attest CLI (verify exit codes 0/2/3/4/5/6) · barrier attack scripts · common/stats library · GitHub Actions. No service, no dashboard — every published number is a file under git.

Two workstreams, one shared library (D-04)

ATTEST needs a rented GPU for about \$2.00 total. BARRIER's kind cluster runs in public CI for £0. Neither blocks the other's ship; both share common/stats only.

System context



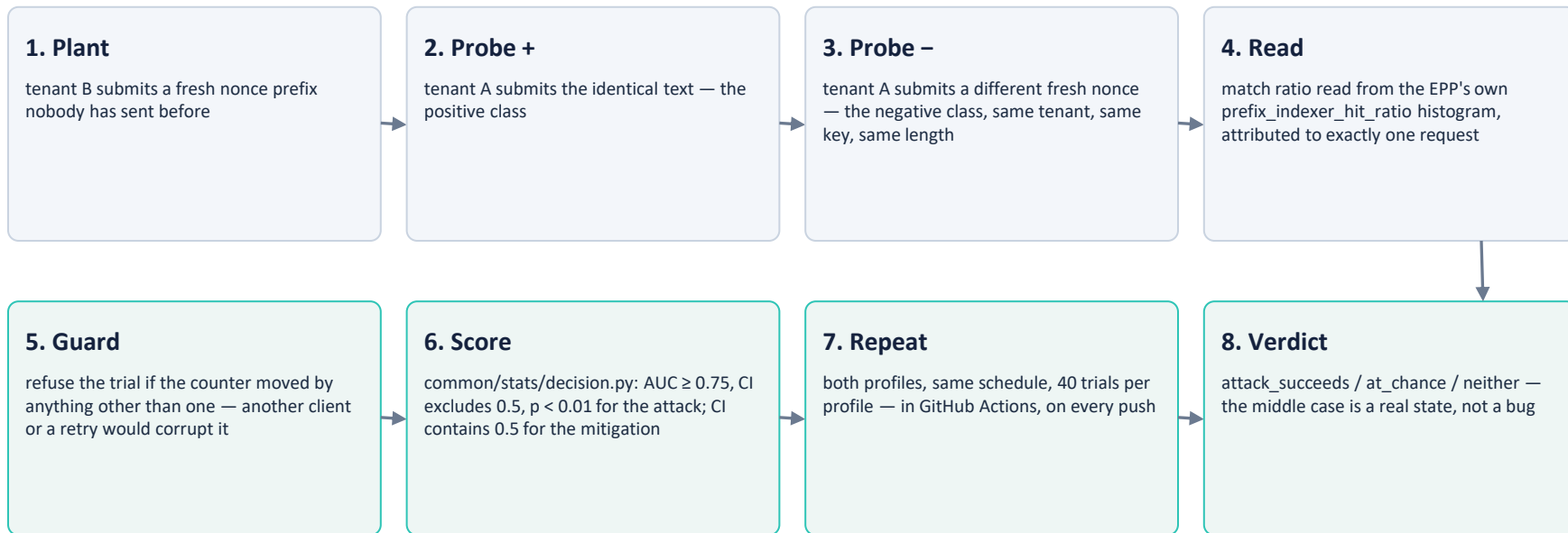
Build/measure-time only: the GPU engine is rented by the hour and released the same day; the kind cluster stands up fresh in CI on every push. Nothing here is an always-on service.

Component architecture



Responsibilities do not overlap (HLD §4): C3 never touches an engine, C1 never computes a statistic, C4 never decides whether a result passes — that is C7's pre-registered test.

The critical flow: measuring the cross-tenant leak



The rule was fixed in LLD §7 before any attack code existed. It has already returned 'neither' once (S-02, ADR-011) and both extremes once (FR-B-03, ADR-012) — never adjusted to fit.

A verdict fixed before the evidence

What the rule requires (`common/stats/decision.py`)

```
attack_succeeds:
    auc >= 0.75
    ci_lower_bound > 0.5
    p < 0.01
at_chance:
    ci contains 0.5
otherwise:
    inconclusive    # neither claim
```

Why it holds up

- one function scores both the attack and the mitigation — nothing to let them silently drift apart
- implemented in `common/stats/` rather than imported: a reviewer can read the whole rule in about forty lines
- bootstrap calibration-tested over 200 null datasets: nominal 95% should cover truth ~95% of the time (91.0% observed)

attack_succeeds

default profile, run #15: AUC 1.0000,
p=9.999e-05

at_chance

hardened profile, run #15: AUC 0.5000, same
schedule

inconclusive

S-02 simulator, run #12: AUC 0.5581
[0.5026,0.6138]

Why this mattered twice

S-02 returned 'ORACLE VIABLE' on a plain millisecond counter — twice — because a five-name ignore-list only excludes noise you already named. The fix was a property test, not a longer list. Two more guards (F-25, F-26) failed the same way before the ground-truth check became tested code.

Data architecture: no database, by design

Raw evidence (C1, C4)

bench/results/<workstream>-<UTC timestamp>-<git SHA>/ — JSONL run output + manifest.json (command, SHA, env, timestamps). Single writer per run-id; immutable once the manifest finalises. Analysis reads, never mutates.

Receipts (C2)

in-toto-style JSON + detached ed25519 signature, JCS-canonicalised so two serialisations produce identical bytes. Model identity anchors to the Hugging Face Hub commit SHA and LFS weight digest — a root the platform team does not control.

Three kinds of file, all under git

raw output
receipts
metrics/headline.json

Figures, generated not drawn

docs/figures/*.svg regenerated by scripts/make_figures.py from bench/results/measurements.json. tests/test_published_numbers.py fails the build if README or the ship report drifts from it.

A database would put a published number behind something that can drift (ADR-001) — the opposite of what a repository whose entire subject is verifiable claims can afford.

Two engines, one plugin, and where the boundary sits

Two engines, one seam (C1)

attest/harness/engine.py defines one client interface; vllm.py and sglang.py implement its lifecycle. SGLang entered as a control arm (ADR-009) to decompose vLLM's confounded cost number — methodology, not coverage.

One plugin, no fork (ADR-002)

llm-d-router exports Register() and a package-level Registry, so barrier/epp/cmd/epp blank-imports the tenant-salt plugin and runs upstream's runner unmodified. Upgrades are a go.mod bump plus a compile check.

Ordering is the whole trust boundary (F-27)

Envoy applies route-level header mutations in the router filter, which runs AFTER ext_proc — a naive strip-and-inject never reaches the EPP. The fix places envoy.filters.http.header_mutation before ext_proc, and overwrites rather than remove-then-add, because Envoy applies removes after adds.

The only 'live' surface is CI

The two-tenant kind topology (llm-d Router + simulator pods) stands up fresh in GitHub Actions on every push to barrier/**, both profiles, £0 — the BARRIER cluster workflow, active and running since 2026-09-07.

Design choices and why

Decision	Alternatives considered	Why this one
Monorepo of CLIs, no services (ADR-001)	Results service + dashboard; one unified CLI	Files under git are the simplest traceability story; a database puts published numbers behind something that can drift
Out-of-tree Go module, no fork (ADR-002)	Fork llm-d-router; configuration-only mitigation	Register/Registry are exported — a droppable plugin is stronger than a fork reviewers discount
Own the AUC/bootstrap/permutation test (ADR-003)	scikit-learn roc_auc_score; statsmodels	A reviewer assessing a security claim should read the forty-line test, not trust a library call
Helm, two values files as the deliverable (ADR-004)	Kustomize overlays; raw YAML per config	The diff between values-default.yaml and values-hardened.yaml IS the argument
ed25519 + in-toto predicate, sigstore deferred (ADR-005)	Sigstore keyless via cosign	Keyless signing needs network + OIDC at verify time, breaking the no-accounts path (D-13)
Strip identity at the proxy, fail closed (ADR-006)	Add the header to llm-d's own InputControlHeaders; trust it as received	Upstream's set is a hardcoded package var; the proxy boundary needs no upstream change
SGLang as ATTEST's control arm (ADR-009)	Publish the confounded vLLM number with a caveat; drop the claim	SGLang runs deterministic with its cache on, supplying the missing 2x2 cell

Major features

Signed attestation receipts

in-toto statement, JCS-canonicalised, ed25519-signed; model identity anchored to a Hugging Face commit SHA and LFS weight digest, not a local hash

Resumable measurement harness

matrix.py × ledger.py: a run that dies at cell 340 of 600 resumes at 341; raw output committed before analysis reads it

Cost-of-determinism decomposition

a 2×2 across vLLM and SGLang isolates determinism's cost (18.0%) from the prefix cache's (a 7.9% penalty on this workload)

Tenant-salt EPP plugin

out-of-tree, registered via llm-d's exported Registry; HMAC-derives a salt from authenticated identity, propagates it to vLLM's own cache too

Two-tenant kind topology in CI

both deployment profiles stood up fresh on every push to barrier/**; the FR-B-03 measurement runs against them for £0

Pre-registered statistics library

AUC, bootstrap CI, permutation test, calibration-tested over 200 null datasets; the decision rule is frozen before any attack code

attest verify CLI

a full exit-code contract: 0 valid, 2 signature invalid, 3 digest mismatch, 4 malformed, 5 Hub unreachable, 6 identity divergent

barrier-diff

the entire mitigation presented as a three-change YAML diff between values-default.yaml and values-hardened.yaml

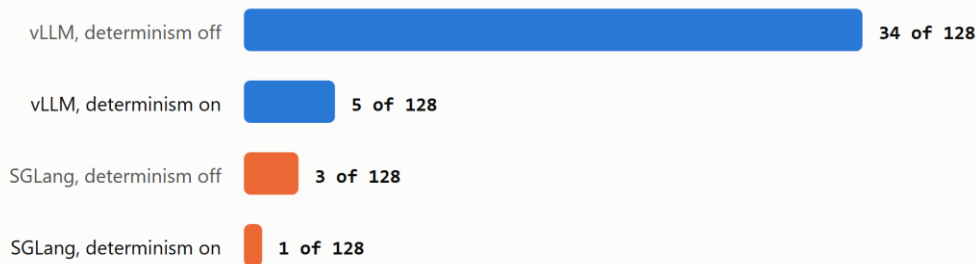
STATE.md findings log

27 findings (F-01...F-27) — the project's own record of what it got wrong and how each was caught

Batched inference is not reproducible

Batched inference is not reproducible at temperature 0

distinct logprob vectors in 128 identical requests — lower is better



The two engines ran under their own defaults and schedulers.
Read as two measurements, not as a controlled comparison.

What this shows

128 identical requests, temperature 0, seed fixed. VLLM_BATCH_INVARIANT=1 cuts 34 distinct outputs to 5 — a large mitigation, not a guarantee. SGLang's deterministic mode reached 1 of 128.

What it does not establish

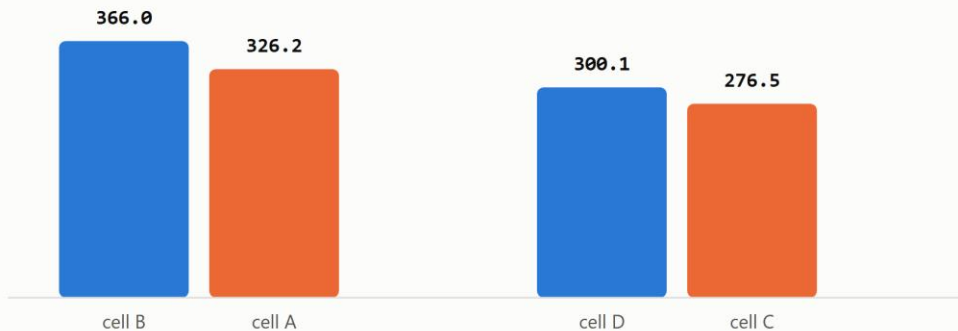
The two engines ran under their own defaults and schedulers — read as two measurements, not a controlled comparison (README).

The cost of determinism, decomposed

The cost of determinism, separated from the cost of losing the cache

SGLang on an H100, mean output tokens/second, 128 trials per cell

■ cache off ■ cache on



D / B 0.820x

determinism alone costs 18.0%, cache held off on both sides

C / A 0.848x

both together — the only comparison a vLLM-shaped experiment can make

The confounded number UNDERSTATES the cost here: the cache is a small penalty on this workload, not a benefit.

Why a second engine

vLLM cannot run batch-invariant with prefix caching on, so its 22.7% number is determinism plus the loss of the cache. SGLang runs deterministic with its cache on, supplying the missing 2x2 cell.

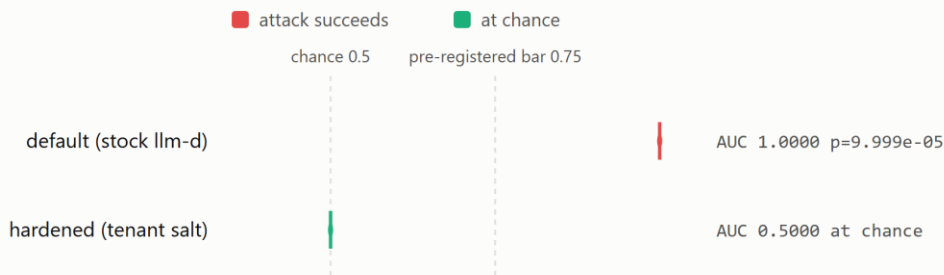
The counter-intuitive part

The confounded figure (0.848x) understates the isolated cost (0.820x) here — the two effects partly cancel on this short-prompt workload.

The cross-tenant leak, and the mitigation closing it

The cross-tenant leak, and the mitigation closing it

same schedule, same pre-registered rule — 40 trials per profile, n=80



Positive class: tenant A probing a prefix tenant B planted. Negative: a prefix nobody planted.

A CONFIRMATION oracle — the probe sends the victim's prompt verbatim, so a perfect match is by construction.

It shows an attacker who can guess a prefix gets it confirmed. It does not show unknown content can be recovered.

Both intervals are degenerate — every observation is identical — so they render as a tick. That is zero variance, not precision.

Same schedule, same rule

40 trials per profile, n=80. default clears the pre-registered bar (AUC 1.0000, p=9.999e-05); hardened lands exactly at chance (AUC 0.5000) — the same instrument, the same threshold.

Scope, stated plainly

A confirmation oracle: the probe sends the victim's prompt verbatim, so 1.0 is by construction. It shows a guessed prefix confirmed, not unknown content recovered.

What a pre-registered rule is actually for

Why the sample size was fixed before the run that counted

S-02 latency channel, AUC with its 95% bootstrap interval



Runs #8 and #11 both sat above chance with an interval straddling it. That is not evidence of no effect; it is what $n=62$ looks like when it cannot resolve an effect of that size. $n=402$ was fixed in the workflow BEFORE the run that got recorded, so the answer could not be reached by stopping when it looked convenient.

Two wrong-shaped answers first

Two runs at $n=62$ put the AUC point estimate above chance with an interval straddling it — not evidence of no effect, just an underpowered sample.

$n=402$ fixed in advance

AUC 0.5581 [0.5026, 0.6138], $p=0.0425$ — clears none of the three thresholds. 'No oracle', stated precisely, not 'no effect'.

Confirming the numbers, and the receipt pipeline

```
$ make headline
uv run python -m metrics.headline
wrote metrics/headline.json: coverage 91.0%, 8 components

$ make card
uv run python -m metrics.headline
wrote metrics/headline.json: coverage 91.0%, 8 components
uv run python metrics/render.py
wrote docs/assets/metrics.svg and README results block

$ make card-check
uv run python metrics/render.py --check
metrics card is current

$ uv run pytest -q
.....F..... [ 21%]
..... [ 43%]
..... [ 65%]
..... [ 86%]
..... [100%]
FAILED tests/attest/test_canonical.py::test_private_key_is_writable_unreadable_to_others
1 failed, 331 passed in 111.45s (0:01:51)
```

```
$ make attest-demo
uv run python scripts/attest_demo.py

run_id      : attest-20260913T000107Z-a324688
run_dir     : bench/results/attest-20260913T000107Z-a324688
cells       : done=1 failed=0
artefacts   : 6 present
OK attest-20260913T000107Z-a324688/c0001
DIGEST MISMATCH: output (subject digest does not match output.token_ids
(declared-'c2cf8ca3261921051fc9c189cd5c1624b9ac73f8ed1838344a56e4914c2ef1f4',
computed-'8a6ae15122001229ed8866f56e342af12ae8187203c3e3b33931743e7c0c48d'))
verify      : OK (valid=0, tampered=3)
cleanup     : demo run directory removed (pass --keep to retain)

NO walking skeleton: PASS
```

Left: make headline / make card / make card-check, then the full pytest run (331 passed, 1 Windows-only failure, 111s). Right: make attest-demo — signs a receipt, then tampers with it three ways and requires the verifier to catch every one ('valid=0, tampered=3').

The mitigation, and proof it compiles

```
$ make barrier-diff
--- barrier/deploy/values-default.yaml
+++ barrier/deploy/values-hardened.yaml
@@ CHANGE 1 of 3 -- declare the tenant-salt plugin @@
plugins:
  - type: prefix-cache-scorer
  - type: queue-scorer
+ - name: tenant-salt
+   type: tenant-salt
+   parameters:
+     identityheader: x-llm-tenant
+     saltsecretfnv: PROVENANCE_SALT_SECRET
+     propagateToTgline: true # CHANGE 2 -- also rewrites vLLM's own cache_salt
+     failClosed: true       # reject rather than route with an empty salt

@@ CHANGE 3 of 3 -- the trust boundary (proxy strips + injects identity) @@
proxy:
  - stripInboundHeaders: []
+ stripInboundHeaders:
+   - x-llm-tenant
+ injectIdentityHeader: x-llm-tenant
```

routing.schedulingProfiles is byte-identical in both files -- the mitigation changes which namespace the hashes fall in, not how endpoints are scored.

```
$ cd barrier/epp && go build ./... && go vet ./... && go test ./... -v
?      github.com/roshanana/provenance/barrier/epp      [no test files]
?      github.com/roshanana/provenance/barrier/epp/cmd/epp      [no test files]
--- PASS: TestApplySaltToMetadataOverTheTetherShouldDead (0.00s)
--- PASS: .../completions (0.00s)
--- PASS: .../chat_completions (0.00s)
--- PASS: .../messages (0.00s)
--- PASS: .../responses (0.00s)
--- PASS: .../conversations (0.00s)
--- PASS: .../embeddings (0.00s)
--- PASS: .../generate (0.00s)
--- PASS: TestEveryBodyVariantWithTheCacheSaltIsCovered (0.00s)
--- PASS: TestApplySaltToOverrideClientSuppliedValue (0.00s)
--- PASS: TestApplySaltToOkOpWithoutASalt (0.00s)
--- PASS: TestRequestHeadersSaltsTheTokenizedPath (0.00s)
--- PASS: TestTwoTenantsDoNotShareNamespacesOnTheTokenizedPath (0.00s)
--- PASS: TestDeriveSaltIsDeterministic (0.00s)
--- PASS: TestDifferentTenantsGetDifferentSalts (0.00s)
--- PASS: TestSaltDependsOnTheSecret (0.00s)
--- PASS: TestSaltLength (0.00s)
--- PASS: TestShortSecretIsRefused (0.00s)
--- PASS: TestEmptyTenantIsRefused (0.00s)
--- PASS: TestTenantIsTrimmed (0.00s)
--- PASS: TestDomainSeparation (0.00s)
--- PASS: TestTenantFromHeadersIsCaseInsensitive (0.00s)
--- PASS: TestMissingHeaderIsRefused (0.00s)
--- PASS: TestEmptyHeaderValueIsRefused (0.00s)
--- PASS: TestUnconfiguredHeaderNameIsError (0.00s)
--- PASS: TestSaltIsHex (0.00s)
PASS
ok      github.com/roshanana/provenance/barrier/epp/plugin    0.222s
```

Left: the entire mitigation as a three-change YAML diff — plugin declaration, salt propagated to vLLM's own cache, proxy strip-and-inject. Right: go build/vet/test against the real llm-d-router v0.10.0 graph — 19 test functions (26 with subtests), all passing.

Security and compliance controls

Boundary	Threat	Control in the code	Evidence
Proxy (trust boundary)	attacker-supplied identity header	strip x-llmd-tenant, inject vouched identity via OVERWRITE_IF_EXISTS_OR_ADD, before ext_proc	run #13: 401 unauthenticated; F-27 fix
EPP prefix index (approximate)	omission / forgery / negligence of client-supplied cache_salt	salt is HMAC-derived from vouched identity; failClosed rejects absent/malformed identity	ADR-012 run #15: default AUC 1.0000, hardened 0.5000
Engine cache (precise path)	salt closes routing index but leaves vLLM's own KV cache shared	plugin rewrites the outbound cache_salt so the engine partitions identically (obligation 3)	Go test: all 7 body variants covered
Salt secret	a compromised secret derives every tenant's salt	min 32 bytes, mounted from a Secret, never logged, never in values files	threat-model.md §8
Receipt verifier	a claim the platform team could fabricate	model identity anchors to the HF Hub commit SHA + LFS digest, a root we do not control	attest verify --online, exit codes 5/6

Residual risks are written down, not hidden: no authentication on the demo cluster beyond the proxy; a compromised salt secret restores every forgery; the client-observable oracle on real vLLM (FR-B-09) is unmeasured.

How it was built: a gated lifecycle



12

ADRs, each with options considered and consequences accepted — several overturning the original brief

27

findings logged in STATE.md (F-01...F-27); four are guards that contained the defect they were written to catch

50

tasks across 7 milestones in the execution plan; 6 detailed packs committed for the earliest waves

\$2.00

total rented GPU spend across six pods for every ATTEST number in this repository

Pre-registered, not post-hoc

BARRIER's success criteria ($AUC \geq 0.75$, CI excludes 0.5, $p < 0.01$) were committed before any attack code existed, and the git history shows that ordering.

One command validates everything

`ruff · ruff format · mypy strict · pytest (332 tests) · Go build/vet/test` — make check. CI is meant to mirror it exactly; a step that passes locally and fails there is treated as the bug.

Quality metrics from this review's own runs



KPI	Value	Source
Python tests	331 passed, 1 failed (Windows-only) of 332	uv run pytest -q
Go tests (barrier/epp)	19 funcs / 26 with subtests, all pass	go build && go vet && go test
ADRs / findings logged	12 / 27 (4 self-catching guards)	decisions.md, STATE.md
Cross-tenant leak (default)	AUC 1.0000, p=9.999e-05, n=80	ADR-012, CI run #15
Mitigation (hardened)	AUC 0.5000, at chance, n=80	ADR-012, CI run #15
Determinism cost, isolated	18.0% (SGLang, D/B = 0.820×)	ADR-009, H100
Determinism cost, confounded	22.7%, 95% CI [0.741, 0.808] (vLLM)	bench/results/cost-h100

The one Python failure asserts a 0600 file-mode; Windows filesystems do not enforce that bit the way POSIX does. Confirmed pre-existing and platform-specific during this review — not a defect in the path CI actually runs (ubuntu-latest).

Quirks and known limitations (stated, not hidden)

A confirmation oracle, not an extraction one

FR-B-03's probe sends the victim's prompt verbatim, so AUC 1.0 is by construction. It shows a guessed prefix gets confirmed; nothing about recovering unknown content (ADR-012).

No client-observable oracle — on the simulator

S-02: latency AUC 0.5581 [0.5026, 0.6138], $p=0.0425$ — clears none of the three thresholds. Real vLLM does vary TTFT with cache state; that question is FR-B-09, open.

Two engines, not a controlled comparison

vLLM and SGLang ran under their own defaults and schedulers. The gap between 34 and 3 (determinism off) is larger than the determinism question alone explains.

No signed receipt against a real engine

attest-demo exercises the full pipeline, tamper detection included, against a stub. The GPU runs measured divergence and cost but emitted no receipts.

Four guards contained the bug they targeted

F-14, F-23, F-25, F-26: a comparison-arm guard that failed open, a millisecond counter mistaken for a discriminator (twice), a ground-truth gate that proved a plugin ran, not that it matched.

CI is disabled; only the cluster workflow runs

The CI workflow is manually disabled on GitHub; BARRIER cluster is the only active workflow, standing up both profiles on every push (confirmed via `gh workflow list --all`).

One pytest failure, Windows-only

`test_private_key_is_written_unreadable_to_others` asserts a 0600 file mode Windows does not express the same way. 331 of 332 pass on this host.

Pre-existing lint/type debt in `metrics/render.py`

ruff found 2 E501 lines and mypy found missing `dict[...]` type args during this review's confirmation run — present before this task, left unfixed as out of scope.

Cost, performance and operability

\$2.00

total rented GPU spend (H100 + A40, six pods) for every ATTEST number in the repository

£0

BARRIER's two-tenant kind cluster, both profiles, stood up fresh in GitHub Actions on every push

111s

the 332-test Python suite on this host (0:01:51); Go build+vet+test completes in well under a second

8

components, no services (HLD \$4) — every artifact is a file under git, traced to a command and a SHA

Operability

- STATE.md is the single source of truth for where the project stands — no repo-wide crawl needed to resume
- bench/results/ keeps the runs that were wrong alongside the ones that were right, each with the reason attached
- make check is the one gate agents, humans and CI all run
- make headline / make card regenerate the offline metrics card and fail the build on drift

What is not yet instrumented

- No alerting, no dashboard, no production deployment target — this is a measurement rig and a shipped mitigation, not a running service
- The GPU session is staged with a human decision point precisely because it is rented hardware, not standing infrastructure
- CI currently runs only the BARRIER cluster workflow (see Quirks)

Roadmap — none of it load-bearing

Next

Real timing oracle

- FR-B-09 on real vLLM, where TTFT does vary with cache state
- The simulator could not answer it (ADR-011) — this is the last open client-observable question

Then

Continuous prefixes

- FR-B-03's separation is binary today (identical or disjoint)
- A shared system prompt with differing tails makes the match ratio continuous and the interval non-degenerate

Then

Receipt on a live engine

- Sign a receipt against real vLLM/SGLang output, not the stub
- Extend the tamper-detection test to that path

Later

Open statistical questions

- Confidence intervals on the SGLang 2x2; SGLang's Triton backend
- Why vLLM's batch-invariant mode leaves 5 residual vectors of 128 where SGLang leaves 1

STATE.md lists all of this explicitly as 'none of it load-bearing' — both workstreams already have their headline result measured.

Appendix A — Decision log (abridged)

ID	Decision	Consequence
ADR-001	Monorepo of CLIs, no services	No runtime to maintain; a dashboard, if ever built, is a mirror, never a source of truth
ADR-002	Out-of-tree Go module, no fork	We track the runner API across releases; upgrades are a go.mod bump plus a compile check
ADR-003	Own the AUC/bootstrap/permutation test	Both the attack and mitigation tests share one implementation — two would make the comparison meaningless
ADR-006	Strip identity at the proxy, fail closed	The threat model states the proxy as the trust boundary; the guarantee is conditional on it
ADR-007	Propagate the salt to the engine, not just the EPP	FR-B-05 gains a third obligation; the residual shrinks to 'whatever survives partitioning both'
ADR-009	SGLang as ATTEST's control arm	D-06 amended: caching pinned off for vLLM because of an upstream integration gap, not determinism itself
ADR-011	No client-observable oracle on the simulator; FR-B-03 rescopes	The attacker-observable oracle becomes FR-B-09's problem, on real vLLM
ADR-012	FR-B-03 measured: the leak exists and the salt closes it	The two-profile diff is now backed by a measurement, not an argument

Appendix B — Build statistics and lessons

Item	Count	Where
ADRs	12	docs/design/decisions.md
Findings logged (F-01...F-27)	27	STATE.md
Findings that were guards containing their own defect	4 (F-14, F-23/24, F-25, F-26)	STATE.md, SHIP-REPORT.md §2
Python tests	332 (331 passing on this host, 1 Windows-only failure)	uv run pytest -q
Go tests (barrier/epp)	19 functions / 26 with subtests, all passing	go test ./... -v
BARRIER cluster CI runs observed	active, both profiles, since 2026-09-07	gh run list -R roshanrana/PROVENANCE

Lessons written into the record

- A limit that has not been re-tested is a guess (ADR-008 — the Go toolchain 'block' was asserted, not tested, and blocked nothing once actually tried).
- A gate whose failure message is silence cannot distinguish the thing it exists to catch (F-25).
- An ignore-list only excludes the noise you already named — S-02 needed a property test, not a longer list (F-23/F-24).
- Four instances of one shape is a pattern worth naming, not four coincidences.

Appendix C — Repository map and how to run

common/stats/	AUC, bootstrap CI, permutation, decision rule
attest/harness/	matrix, ledger, engine client, vLLM + SGLang
attest/receipt/	in-toto schema, JCS, ed25519, verify CLI
attest/analysis/	divergence tables, cost-of-determinism CIs
barrier/attack/	S-02 spike, FR-B-03 probe, ground truth
barrier/epp/	Go: tenant-salt plugin + custom EPP binary
barrier/deploy/	kind + Helm: values-default vs values-hardened
bench/results/	immutable raw output, every number's source
metrics/	headline.json, render.py (drift-checked)
docs/design/	requirements, HLD, LLD, decisions, spikes
docs/tasks/	early task packs (waves 1-2)
.github/workflows/	ci.yml (disabled), barrier.yml (active)

Run it

```
uv sync
make check                # 332 Python
                           # + Go, ~2 min
make attest-demo          # full pipeline,
                           # incl. tamper test
make barrier-diff         # the mitigation,
                           # as a diff

# needs Docker + kind, no GPU
make barrier-up PROFILE=hardened
make barrier-spike

# needs a real GPU
make attest-stage1 \
    ENGINE_URL=http://host:8000
```

Questions

PROVENANCE · roshanrana/PROVENANCE · design docs, decision log and bench/results in the repository